



https://github.com/SETAPESU26/14_2048. i need to execute and submit deliverables

I checked the assigned repository. It is **Scenario 14 — 2048**, and the README specifies **4 tasks**:

1. **Task 1:** Fix 2048 merge semantics.
2. **Task 2:** Add win detection, no-move detection, and correct tile creation.
3. **Task 3:** Add one-level undo + best score.
4. **Task 4:** Add move/merge feedback. GitHub

The required testing includes four-equal tiles, unchanged moves, 2048 detection, game-over detection, undo, score restoration, invalid commands, and quitting. GitHub

Your Lab 4 submission

Your Lab-4 folder should ultimately contain:

Plain text

```
Lab-4/  
├─ Before_Video.mp4  
├─ After_Video.mp4  
├─ 14_2048_Updated.zip  
└─ Chat_History.pdf
```

The lab handout requires **before/after 10-second videos, updated code, and exported chat history**. Lab_4_VibeCoding_Student_handout

Also, the repository specifically requires **separate commits for each task** and says not to create a PR to SETAPESU26 .

We should do it in this order

Step 1 — Clone the repository

Bash

```
git clone https://github.com/SETAPESU26/14_2048.git  
cd 14_2048
```

Step 2 — Run the original/broken code

Bash

```
python3 main.py
```

The README says no package installation is required.  GitHub

Step 3 — Record the BEFORE video

Record about **10 seconds** showing the original broken behavior, especially the merge bug.

Step 4 — Complete Task 1 → commit

```
</> Bash

git add .
git commit -m "Task 1: Fix merge semantics"
```

Step 5 — Complete Task 2 → commit

```
</> Bash

git add .
git commit -m "Task 2: Add game state handling"
```

Step 6 — Complete Task 3 → commit

```
</> Bash

git add .
git commit -m "Task 3: Add undo and best score"
```

Step 7 — Complete Task 4 → commit

```
</> Bash

git add .
git commit -m "Task 4: Add move feedback"
```

Step 8 — Test everything

Then record the **AFTER video** showing the completed features.

Step 9 — Push to your personal GitHub repository

You should push to **your own fork/personal repo**, not directly to SETAPESU26 . The lab instructions explicitly say to clone/fork, make changes, push to your personal account, and not

raise a PR to the main repo.  Lab_4_VibeCoding_Student_handout

Important: Don't start changing the code blindly. The README says to inspect the existing code and reproduce the original defect first.  GitHub

If you want, **we can start with Step 1 now**. I can give you the exact Mac terminal commands, then we'll run the original 2048 and make the **before-video** correctly.

I am working on Task 1 of the Lab 4 Vibe Coding assignment for the 2048 repository.

Please inspect the existing `board.py` implementation before making changes.

Task 1 requirement:

Fix the merge algorithm so that an original tile can participate in at most one merge during a single move. Preserve the existing compression and movement behavior.

The important edge case is four equal tiles:

`[2, 2, 2, 2]` should become `[4, 4, 0, 0]`, not `[8, 0, 0, 0]`.

Please:

1. Explain the current bug in `slide_line()`.
2. Provide the minimal code change needed to fix Task 1.
3. Do not implement Tasks 2, 3, or 4 yet.
4. Keep the existing project structure and avoid adding dependencies.
5. Give me test cases for Task 1, including four equal tiles, separated equal pairs, and ordinary slides.

I inspected the actual `board.py` from the assigned `SETAPESU26/14_2048` repository. The file is only 75 lines and the bug is explicitly in `slide_line()`.  GitHub

1. Current bug in `slide_line()`

The current implementation is:

 Python

```
@staticmethod
def slide_line(line):
```

```

values = [x for x in line if x]
result = []

for value in values:
    if result and result[-1] == value:
        result[-1] *= 2 # intentional double-merge bug
    else:
        result.append(value)

return result + [0] * (SIZE - len(result))

```

The problem is that after two equal tiles merge, the newly created tile remains available for another merge during the **same loop**.

For example:

</> Plain text

[2, 2, 2, 2]

Current behavior:

</> Plain text

```

2 → result = [2]
2 → [4]
2 → [8]
2 → [8, 2]

```

So it can incorrectly produce:

</> Plain text

[8, 2, 0, 0]

The correct 2048 behavior is:

</> Plain text

```

[2, 2, 2, 2]
  ↓
[4, 4, 0, 0]

```

The important rule is: **once a tile has been created by a merge, it cannot merge again during that same move.**

2. Minimal Task 1 fix

We only need to change `slide_line()` .

Replace the existing function with:

`</>` Python

```
@staticmethod
def slide_line(line):
    values = [x for x in line if x]
    result = []
    i = 0

    while i < len(values):
        if i + 1 < len(values) and values[i] == values[i + 1]:
            result.append(values[i] * 2)
            i += 2
        else:
            result.append(values[i])
            i += 1

    return result + [0] * (SIZE - len(result))
```

What changed?

Instead of processing one value at a time and comparing it with the **already-created result**, we process the compressed line in pairs.

For:

`</>` Plain text

```
[2, 2, 2, 2]
```

The algorithm does:

`</>` Plain text

```
i = 0
2 == 2 → append 4
i = 2

2 == 2 → append 4
i = 4
```

Result:

</> Plain text

```
[4, 4, 0, 0]
```

This is a **Task 1-only change**. It does not modify `move_left()` , `move_right()` , `move_up()` , `move_down()` , `can_move()` , scoring, win detection, undo, or feedback. Those remain for later tasks. [GitHub](#)

3. Task 1 test cases

Because `slide_line()` is a `@staticmethod` , we can test it directly without changing the project structure.

You can temporarily run:

</> Bash

```
python3 -c "from board import Board; print(Board.slide_line([2,2,2,2]))"
```

Expected:

</> Plain text

```
[4, 4, 0, 0]
```

Test 1 — Four equal tiles

</> Plain text

Input:

```
[2, 2, 2, 2]
```

Expected:

```
[4, 4, 0, 0]
```

This is the most important Task 1 test.

Test 2 — Three equal tiles

</> Plain text

Input:

[2, 2, 2, 0]

Expected:

[4, 2, 0, 0]

The first two 2 s merge, while the third 2 remains separate.

Test 3 — Separated equal pairs

</> Plain text

Input:

[2, 2, 4, 4]

Expected:

[4, 8, 0, 0]

Both pairs merge independently.

Test 4 — Ordinary slide

</> Plain text

Input:

[2, 0, 0, 4]

Expected:

[2, 4, 0, 0]

No merge occurs; tiles simply compress toward the left.

Test 5 — No merge

</> Plain text

Input:

[2, 4, 8, 16]

Expected:
[2, 4, 8, 16]

Test 6 — Mixed values

</> Plain text

Input:
[2, 2, 4, 8]

Expected:
[4, 4, 8, 0]

Test 7 — Another important merge case

</> Plain text

Input:
[4, 4, 4, 4]

Expected:
[8, 8, 0, 0]

Not:

</> Plain text

[16, 0, 0, 0]

4. Test `move_right()` too

Because `move_right()` uses `slide_line()` in reverse, the same fix should work there:

</> Plain text

Input row:
[2, 2, 2, 2]

Move right:

```
[0, 0, 4, 4]
```

The existing `move_right()` implementation already reverses the line, calls `slide_line()`, and reverses it back, so **do not modify it**. [🔗 GitHub](#)

5. After making the change

Run:

```
</> Bash
```

```
python3 -c "from board import Board; print(Board.slide_line([2,2,2,2])); print(Board.slide_line([
```

You should get:

```
</> Plain text
```

```
[4, 4, 0, 0]
[4, 8, 0, 0]
[2, 4, 0, 0]
[8, 8, 0, 0]
```

Then make your Task 1 commit

```
</> Bash
```

```
git status
git diff
git add board.py
git commit -m "Task 1: Fix 2048 merge semantics"
```

And verify:

```
</> Bash
```

```
git log --oneline -1
```

You should see:

```
</> Plain text
```

Task 1: Fix 2048 merge semantics

Do not implement Tasks 2–4 yet. This change is sufficient for Task 1 while preserving the existing movement/compression structure.

yes i did it

Great 👍 **Task 1 is completed.**

Before moving to Task 2, make sure your Task 1 commit exists:

```
</> Bash
```

```
git log --oneline -1
```

It should show something like:

```
</> Plain text
```

```
Task 1: Fix 2048 merge semantics
```

Also run:

```
</> Bash
```

```
git status
```

Ideally:

```
</> Plain text
```

```
nothing to commit, working tree clean
```

Next: Task 2

We'll work **only on Task 2** and won't touch Tasks 3 or 4.

Task 2 will cover:

- Correct tile creation after a move
- Win detection when reaching 2048
- No-move/game-over detection
- Testing the required edge cases

When you're ready, say "**Task 2**", and I'll give you the exact changes step by step.

Task 2 Prompt — 2048 Game-State Handling

I have completed Task 1 of the Lab 4 Vibe Coding assignment for the 2048 repository. Now I want to implement **Task 2 only**.

Please inspect the current code **after my Task 1 changes** before making any modifications.

Task 2 requirements:

1. Detect when the player reaches **2048** and handle the win state correctly.
2. Detect when the board has **no legal moves remaining** and handle the game-over state correctly.
3. Do **not** add a new random tile when the player makes a move that does not actually change the board.
4. Preserve the Task 1 merge behavior. An original tile must still participate in at most one merge per move.

Please:

- Explain the current game-state problems in the existing code.
- Make the **minimum necessary changes** to implement Task 2.
- Do not implement Task 3 (Undo/Best Score) or Task 4 (Move Feedback) yet.
- Do not undo or modify the Task 1 fix.
- Keep the existing project structure.
- Do not add unnecessary dependencies.
- Provide the exact files/functions that need to be changed.
- Provide the complete corrected code for each changed function.
- Give me clear test cases for:
 - reaching 2048,
 - a board with no legal moves,
 - an unchanged/invalid move,
 - a valid move that should create exactly one new tile,

- and confirming Task 1 still works.

I inspected the upstream `board.py`, `game.py`, and README. The README confirms Task 2 is specifically **win detection, no-move detection, and correct tile-creation semantics**, while Task 3/4 must remain untouched.  [GitHub](#)

One limitation: I can inspect the upstream repository, but I **cannot see your local Task 1-edited files** unless you upload them or paste them here. So the patch below assumes you made exactly the Task 1 `slide_line()` change we used earlier. The upstream repository still shows the old merge code.  [GitHub](#)

What is currently wrong

1. Win detection

Interestingly, the current `game.py` already checks:

 Python

```
if any(2048 in row for row in self.board.grid):  
    print("You reached 2048!")  
    return
```

So the basic **2048 detection is already present**.  [GitHub](#)

We should **not unnecessarily rewrite it**.

2. No-move detection

`Board.can_move()` already correctly checks:

- whether there is an empty cell;
- whether two horizontally adjacent tiles are equal;
- whether two vertically adjacent tiles are equal.

If none are possible, it returns `False`.  [GitHub](#)

And `game.py` already uses:

 Python

```
if not self.board.can_move():  
    print("No legal moves remain.")  
    return
```

So again, this part is already implemented in the starter.  [GitHub](#)

3. The actual Task 2 bug: tile creation

This is the important part.

Current `Game.move()` does:

`</>` Python

```
changed = moves[key]()

if changed:
    self.board.add_random_tile()

return changed
```

This is actually the **correct basic behavior**: a new tile is created only when the move changes the board. [GitHub](#)

Therefore, based on the current upstream code, **there is no need to modify `Game.move()` for this requirement.**

The `move_left()`, `move_right()`, `move_up()`, and `move_down()` functions already return `changed`, based on comparing the old line with the new line. [GitHub](#)

So what should you actually change?

This is important for your lab: **don't make unnecessary changes just to create a Task 2 commit.**

The current starter already contains the three required game-state mechanisms at a basic level:

`</>` Plain text

```
2048 reached      → detected
No legal moves    → detected
Unchanged move    → no new tile
```

The README nevertheless lists these under Task 2, so your task should be to **verify the behavior and make only a minimal correction if your Task 1 branch exposes a problem.** [GitHub](#)

The functions involved

File	Function	Task 2 role
<code>board.py</code>	<code>move_left()</code>	Determines whether move changed board

File	Function	Task 2 role
board.py	move_right()	Determines whether move changed board
board.py	move_up()	Determines whether move changed board
board.py	move_down()	Determines whether move changed board
board.py	can_move()	Detects whether legal moves remain
game.py	move()	Adds tile only after successful move
game.py	run()	Detects 2048/game over

These are already present in the repository.  GitHub +1

Minimal recommendation

Do not change `board.py` beyond your Task 1 `slide_line()` fix.

Keep your Task 1 version of:

 Python

```
@staticmethod
def slide_line(line):
    values = [x for x in line if x]
    result = []
    i = 0

    while i < len(values):
        if i + 1 < len(values) and values[i] == values[i + 1]:
            result.append(values[i] * 2)
            i += 2
        else:
            result.append(values[i])
            i += 1

    return result + [0] * (SIZE - len(result))
```

That preserves the Task 1 requirement.

Test Task 2 properly

Because random tile placement makes visual testing inconvenient, use controlled boards.

Create a temporary test file:

 Bash

```
touch test_task2.py
```

Put this in it:

 Python

```
from board import Board
from game import Game

# 1. Test reaching 2048
game = Game()
game.board.grid = [
    [2048, 0, 0, 0],
    [0, 0, 0, 0],
    [0, 0, 0, 0],
    [0, 0, 0, 0]
]

assert any(2048 in row for row in game.board.grid)
print("PASS: 2048 detection")

# 2. Test no legal moves
board = Board()
board.grid = [
```

Run:

 Bash

```
python3 test_task2.py
```

Expected:

 Plain text

```
PASS: 2048 detection
PASS: no-move detection
PASS: unchanged move does not change board
PASS: valid move creates exactly one new tile
PASS: Task 1 four-tile merge
```

Important detail about Test 4

For:

</> Plain text

```
[2, 0, 0, 0]
```

moving right produces:

</> Plain text

```
[0, 0, 0, 2]
```

The number of empty cells goes from **15** → **14** after the move and new tile creation. That confirms exactly one new tile was added.

Then Task 2 commit

Once those tests pass:

</> Bash

```
rm test_task2.py
```

Then:

</> Bash

```
git status
```

If you **didn't actually modify any production files**, don't create a fake Task 2 commit. Your repository already has these Task 2 behaviors.

However, your assignment expects **separate commits for each task**, so if your instructor expects an actual Task 2 implementation commit, I need to see **your current `game.py` and `board.py` after Task 1** before recommending a legitimate minimal change.

Best next step

Run:

</> Bash

```
cat board.py
```

and:

```
</> Bash
```

```
cat game.py
```

Paste both outputs here.

Then I can compare **your actual Task 1 state** and give you the exact Task 2 patch without accidentally overwriting your Task 1 work or implementing Task 3/4.

Task 3 — Undo + Best Score

Copy and paste this as your **Task 3 prompt**:

I have completed Tasks 1 and 2 of the Lab 4 Vibe Coding assignment for the 2048 repository. Now I want to implement **Task 3 only**.

Please inspect the current code first and preserve all working changes from Tasks 1 and 2.

Task 3 requirements:

1. Add a **one-level Undo** feature.
2. Undo must restore the board to exactly its state before the previous valid move.
3. Undo must also restore the **score** to the value before that move.
4. Undo should restore both the board and score together, not just the board.
5. Track and display the **best score for the current run**.
6. The best score should not decrease when the player uses Undo.
7. Do not allow Undo to incorrectly undo multiple moves; only one previous state needs to be stored.
8. Preserve all Task 1 merge behavior and Task 2 game-state handling.

Please:

- Explain how the current code handles moves and score.
- Explain where the previous board and score should be stored.
- Make only the changes necessary for Task 3.
- Do not implement Task 4 yet.
- Do not add unnecessary dependencies.
- Keep the existing project structure.
- Give me the exact files/functions that need to be changed.
- Provide the corrected code.
- Give me test cases for:
 - making a move and undoing it,
 - verifying the board is restored,
 - verifying the score is restored,
 - verifying only one undo is available,
 - verifying best score is maintained,
 - and confirming Tasks 1 and 2 still work.

Yes. For **Task 3**, the safest approach is to preserve your Task 1 and Task 2 changes and add only **one saved board + score state** and **best score**.

I can give you the exact implementation, but because you asked me to **inspect your current code first**, please paste your current:

```
</> Bash
```

```
cat board.py
```

and

```
</> Bash
```

```
cat game.py
```

from your repository **after Tasks 1 and 2**.

I don't want to give you a patch based on the original repository and accidentally overwrite your Task 1/2 work.

Once you paste those two files, I'll give you:

- exact changes to `board.py` / `game.py`

- complete corrected functions
- one-level undo only
- board **and** score restoration
- best-score tracking that survives Undo
- exact test commands
- the Task 3 commit command

We will **not touch** Task 4.

Task 4 — Move Feedback

After Task 3 is working and committed, use this as your **Task 4 prompt**:

I have completed Tasks 1, 2, and 3 of the Lab 4 Vibe Coding assignment for the 2048 repository. Now I want to implement **Task 4 only**.

Please inspect the current code first and preserve all existing functionality from Tasks 1, 2, and 3.

Task 4 requirements:

1. Add concise feedback for a **successful/accepted move**.
2. If tiles merge, the feedback should indicate that a merge occurred and provide useful information such as the merged value.
3. Do **not** report a successful move when the user's input is invalid or the board does not change.
4. One accepted move should produce **one action-level result**, rather than multiple confusing messages for the same move.
5. Keep the feedback concise and suitable for the existing terminal-based 2048 game.
6. Preserve the Task 1 merge semantics.
7. Preserve Task 2 win/game-over and unchanged-move handling.
8. Preserve Task 3 Undo and Best Score functionality.

Please:

- Inspect the current move-processing flow before changing anything.

- Explain where the feedback should be generated.
- Make only the changes necessary for Task 4.
- Do not rewrite unrelated parts of the project.
- Do not add unnecessary dependencies.
- Keep the existing project structure.
- Provide the exact files/functions that need to be changed.
- Provide the corrected code.
- Give me test cases for:
 - a successful move,
 - a successful move with a merge,
 - an unchanged move,
 - invalid input,
 - game-over/win situations,
 - and confirming Undo and Best Score still work.

Yes — that is the correct **Task 4 prompt**. Use it only after Task 3 has been implemented, tested, and committed.

For Task 4, the important constraint is that feedback should be generated at the **move-processing level**, so one accepted move produces one concise result and invalid/unchanged moves produce no success message.

Task 4 expected behavior

Situation	Expected feedback
Valid move, no merge	Concise success message
Valid move + merge	Success + merged value
Invalid command	No success message
Valid command but board unchanged	No success message
Win	Preserve existing win handling
Game over	Preserve existing game-over handling
Undo	Still works
Best score	Still works

Situation**Expected feedback**

[2,2,2,2]

Still becomes [4,4,0,0]

For example, acceptable terminal feedback could be:

```
</> Plain text
```

```
Move accepted.
```

and for a merge:

```
</> Plain text
```

```
Move accepted – merged to 4.
```

The exact wording can remain simple; the important part is **one action-level message per accepted move**.

Before you start Task 4

Make sure Task 3 is committed:

```
</> Bash
```

```
git status
```

Then:

```
</> Bash
```

```
git log --oneline -4
```

You should have separate commits for Tasks 1, 2, and 3.

Don't modify anything for Task 4 yet if Task 3 isn't working. When you are ready, paste your current `board.py` and `game.py` after Task 3, and I can inspect the actual code and give you the minimal Task 4 change.

yes i did it

Perfect 🍌 If **Task 4 is completed and committed**, you are now at the final stage of the Lab 4 assignment.

1. Verify all 4 commits

Run:

```
</> Bash
```

```
git log --oneline --all -5
```

You should have separate commits similar to:

```
</> Plain text
```

```
Task 4: Add move feedback  
Task 3: Add undo and best score  
Task 2: Add game-state handling  
Task 1: Fix 2048 merge semantics
```

Then:

```
</> Bash
```

```
git status
```

You want:

```
</> Plain text
```

```
nothing to commit, working tree clean
```

2. Run the final game

From the 14_2048 folder:

```
</> Bash
```

```
python3 main.py
```

Test the important features:

- 2 2 2 2 → merging should follow **4, 4**, not 8.
- Reach 2048 → win is detected.
- Full board with no possible merge → game over.
- Invalid/unchanged move → no new tile and no false success feedback.
- Valid move → exactly one new tile.

- Undo → restores **both board and score**.
- Second undo → should not undo another move.
- Best score → should remain even after Undo.
- Successful merge → concise merge feedback appears.

3. Push your final repository

If your personal GitHub repository is already configured:

 Bash

```
git push origin main
```

If your branch is `master` :

 Bash

```
git push origin master
```

Check the remote with:

 Bash

```
git remote -v
```

Do not create a PR to SETAPESU26 , as the lab instructions explicitly prohibit that.

 [Lab_4_VibeCoding_Student_handout](#)

4. Create your Lab-4 submission folder

Your final structure should be:

 Plain text

```
Lab-4/  
├─ Before_Video.mp4  
├─ After_Video.mp4  
├─ 14_2048_Updated.zip  
└─ Chat_History.pdf
```

The handout specifically requires these three categories: **before/after videos, updated code, and exported chat history.**  [Lab_4_VibeCoding_Student_handout](#)

5. Create the ZIP

From the directory containing 14_2048 :

 Bash

```
zip -r 14_2048_Updated.zip 14_2048
```

Don't include unnecessary things such as:

 Plain text

```
.git/  
__pycache__/  
.DS_Store
```

A cleaner command is:

 Bash

```
zip -r 14_2048_Updated.zip 14_2048 \  
-x "14_2048/.git/*" \  
"14_2048/**/__pycache__/*" \  
"14_2048/**/*.*pyc" \  
"14_2048/**/.DS_Store"
```

6. Before/After videos

Before video: approximately 10 seconds showing the original broken implementation.

After video: approximately 10 seconds showing your completed 2048 implementation.

The lab explicitly asks for a 10-second recording before changes and another after all changes.

 Lab_4_VibeCoding_Student_handout

7. Chat history

Export this ChatGPT conversation as a PDF and name it:

 Plain text

```
Chat_History.pdf
```

This should show your Vibe Coding interaction/prompts and the implementation process.

Final checklist

</> Plain text

- ☑ Task 1 completed + separate commit
- ☑ Task 2 completed + separate commit
- ☑ Task 3 completed + separate commit
- ☑ Task 4 completed + separate commit
- ☑ Final code pushed to personal GitHub
- ☑ Before video
- ☑ After video
- ☑ Updated code ZIP
- ☑ Chat history PDF
- ☑ Lab-4 folder created
- ☑ No PR to SETAPESU26

Your coding portion is now complete. The remaining work is preparing the **videos, ZIP, Chat History PDF, and Lab-4 GitHub folder.**

I have completed Tasks 1, 2, and 3 in my local 2048 repository. I now need to implement **Task 4 only**.

First inspect my **actual current local game.py , board.py , and main.py** . Do not assume that functions such as `process_move()` or `slide_and_merge()` exist.

Task 4 requirements:

1. Give concise feedback after a successful move.
2. If the move contains a merge, report the merge/result concisely.
3. If the selected direction does not change the board, report that no tiles moved.
4. If the user enters an invalid key, report that the input is invalid.
5. Produce only **one action-level feedback message per user action**.
6. Do not add a new tile or modify the score for an unchanged/invalid move.
7. Preserve all Task 1 merge behavior.
8. Preserve all Task 2 win/game-over behavior.
9. Preserve all Task 3 Undo and Best Score behavior.
10. Do not implement or redesign anything outside Task 4.
11. Do not add unnecessary dependencies.

Before giving code, explain exactly how the existing `Game.move()` and `Game.run()` flow works.

Then provide the **minimum changes to my actual existing files**, not a hypothetical implementation.

Give me the complete corrected code for only the functions/files that actually need modification.

Finally, provide manual test cases for:

- valid move without merge,
- valid move with merge,
- unchanged move,
- invalid input,
- 2048 win,
- game over,
- Undo,
- Best Score.

Do not modify Tasks 1–3.

Yes — I'll do this against your **actual Task 3 code**, not the upstream repository.

Right now, I don't have your local `game.py`, `board.py`, and `main.py` available in this chat/runtime, so I can't honestly inspect `Game.move()` and `Game.run()` yet.

Please run these commands from your `14_2048` folder and paste the output:

```
</> Bash
```

```
cat game.py
```

```
</> Bash
```

```
cat board.py
```

```
</> Bash
```

```
cat main.py
```

Or, even better, **upload the three `.py` files here**.

Once you provide them, I'll:

1. Trace the **actual** `Game.move()` → `Game.run()` flow.
2. Identify exactly where feedback belongs.
3. Preserve your existing **Tasks 1–3** implementation.
4. Modify only the functions/files actually required for Task 4.
5. Ensure **one feedback message per user action**.
6. Ensure invalid/unchanged moves don't add tiles or change score.
7. Give you the complete corrected code.
8. Give you the manual test cases for all 8 scenarios you listed.
9. Give you the exact Task 4 commit command.

I will **not assume** `process_move()` , `slide_and_merge()` , or any other function exists.